

TypeScript

TypeScript = JavaScript + Types



Svetlin Nakov, PhD

Co-founder @ SoftUni



SoftUni AI

<https://ai.softuni.bg>

Agenda

1. Intro to **TypeScript** (TS)
 - Using Simple Types in TypeScript
 - Compiling and Running TS: **tsc** and **tsx**
2. **Functions** and **Objects** in TypeScript
 - Typed Functions, Type Declarations, Interfaces, Typed Arrays, Tuples
3. Case Study: **Validating Invoices**
4. **Unions, Literals, Enums** in TypeScript
5. **Classes & Generics** in TypeScript
6. TypeScript and **Vite**: Component-based Web Projects



Sli.do Code

#AI-Programming

Join at

slido.com

#AI-Programming



Breaks

20:00 / 21:00



Intro to TypeScript

Simple Types. Installing and Running TypeScript



What is TypeScript?

- **TypeScript** = JavaScript + types
- **TypeScript** (TS) is a superset of JavaScript
 - Adds **static typing**: `let age: number = 25;`
 - Higher **code scalability** (for bigger projects)
 - TS **catches many bugs** at compile time
 - Adds **data types**: types, interfaces, enums, tuples
 - Adds full **OOP** (object-oriented programming) support
 - Compiles down to JavaScript with a **build tool** (esbuild, vite)



Static Typing Avoids Bugs

- **JavaScript** code is more **error prone** (no type checking)

```
function add(a, b) {  
    return a + b;  
}  
add("5", 10); // → "510" (runtime surprise!)
```



- **TypeScript** catches bugs before running the code:

```
function add(a: number, b: number): number {  
    return a + b;  
}  
add("5", 10); // ← Compile-time error!
```



Installing and Running TypeScript

- **Installing TypeScript** (the TS compiler):  TypeScript

```
npm -g install typescript
```

```
tsc --version
```

- **Running** the TypeScript compiler (transpilation):

```
tsc example.ts
```



```
example.js
```



Compiling and Running TypeScript

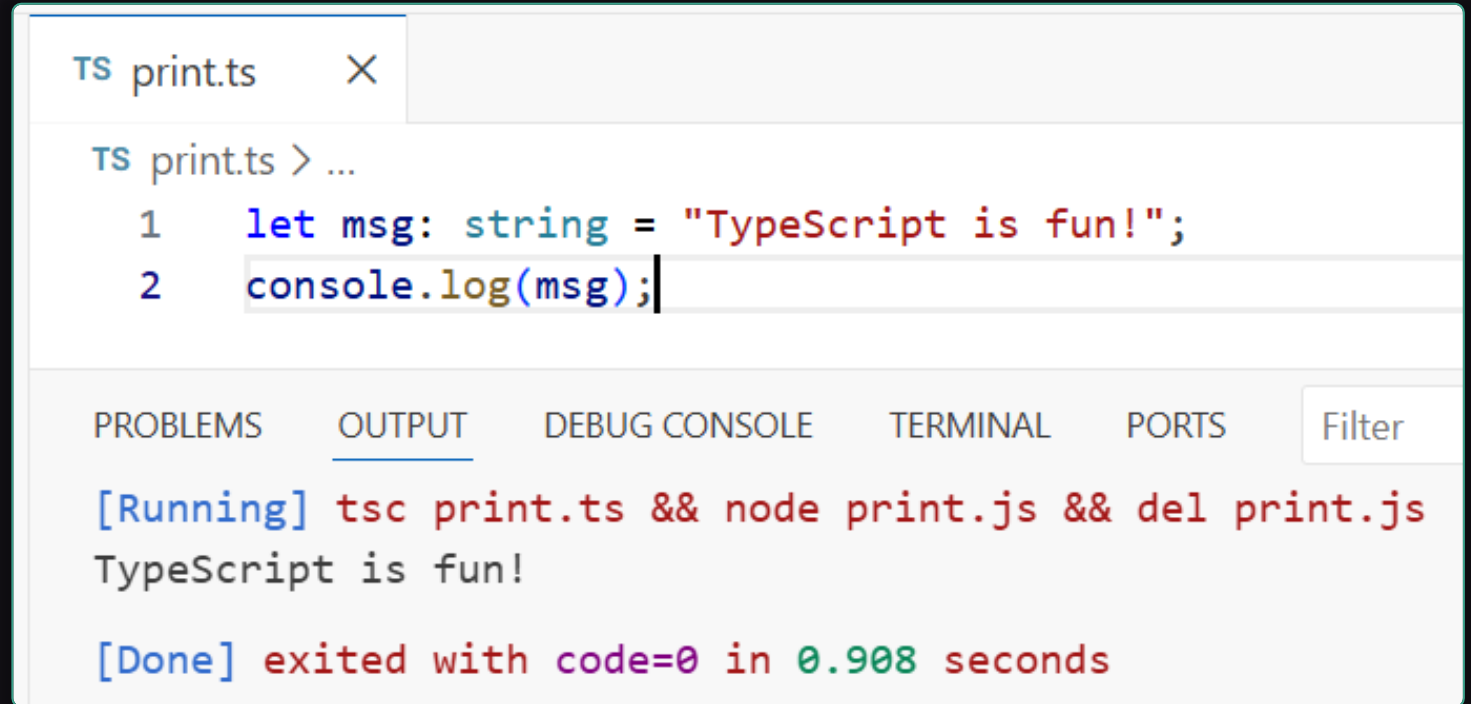
- **Compiling, running** TS code + **cleanup**

- Windows

```
tsc example.ts  
node example.ts  
del example.ts
```

- Linux / Mac

```
tsc example.ts  
node example.ts  
rm example.ts
```



The screenshot shows a VS Code editor window with a file named 'print.ts'. The code in the editor is:

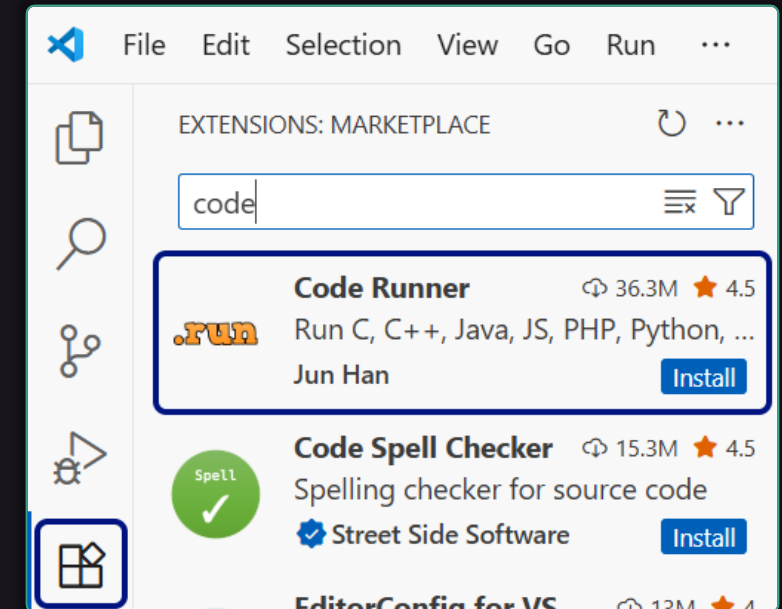
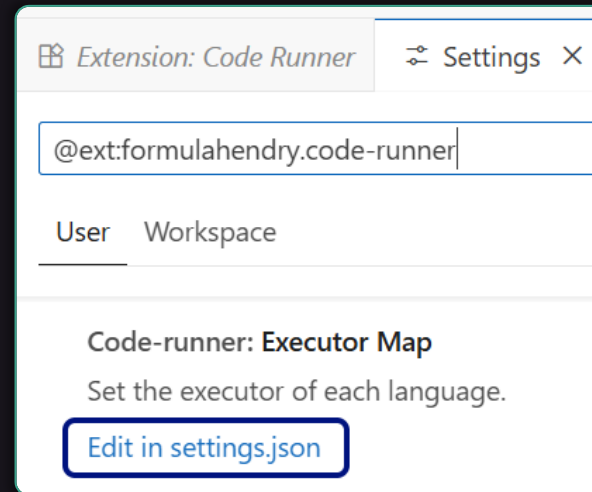
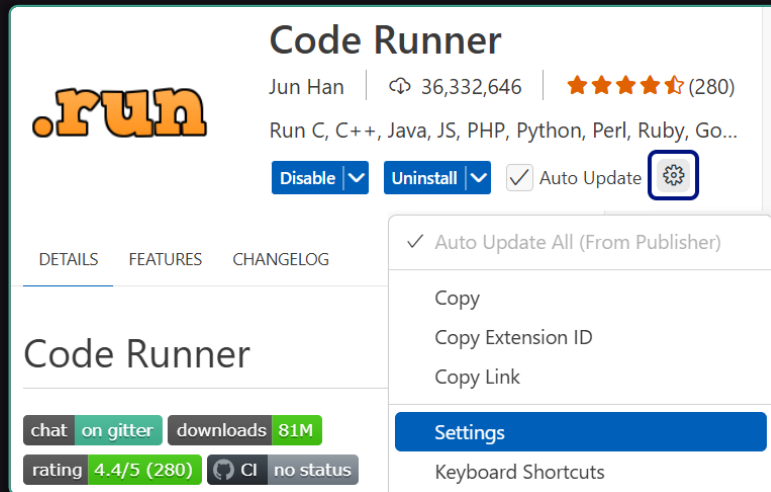
```
1 let msg: string = "TypeScript is fun!";  
2 console.log(msg);
```

Below the editor, the 'OUTPUT' panel is open, showing the command executed and its output:

```
[Running] tsc print.ts && node print.js && del print.js  
TypeScript is fun!  
[Done] exited with code=0 in 0.908 seconds
```

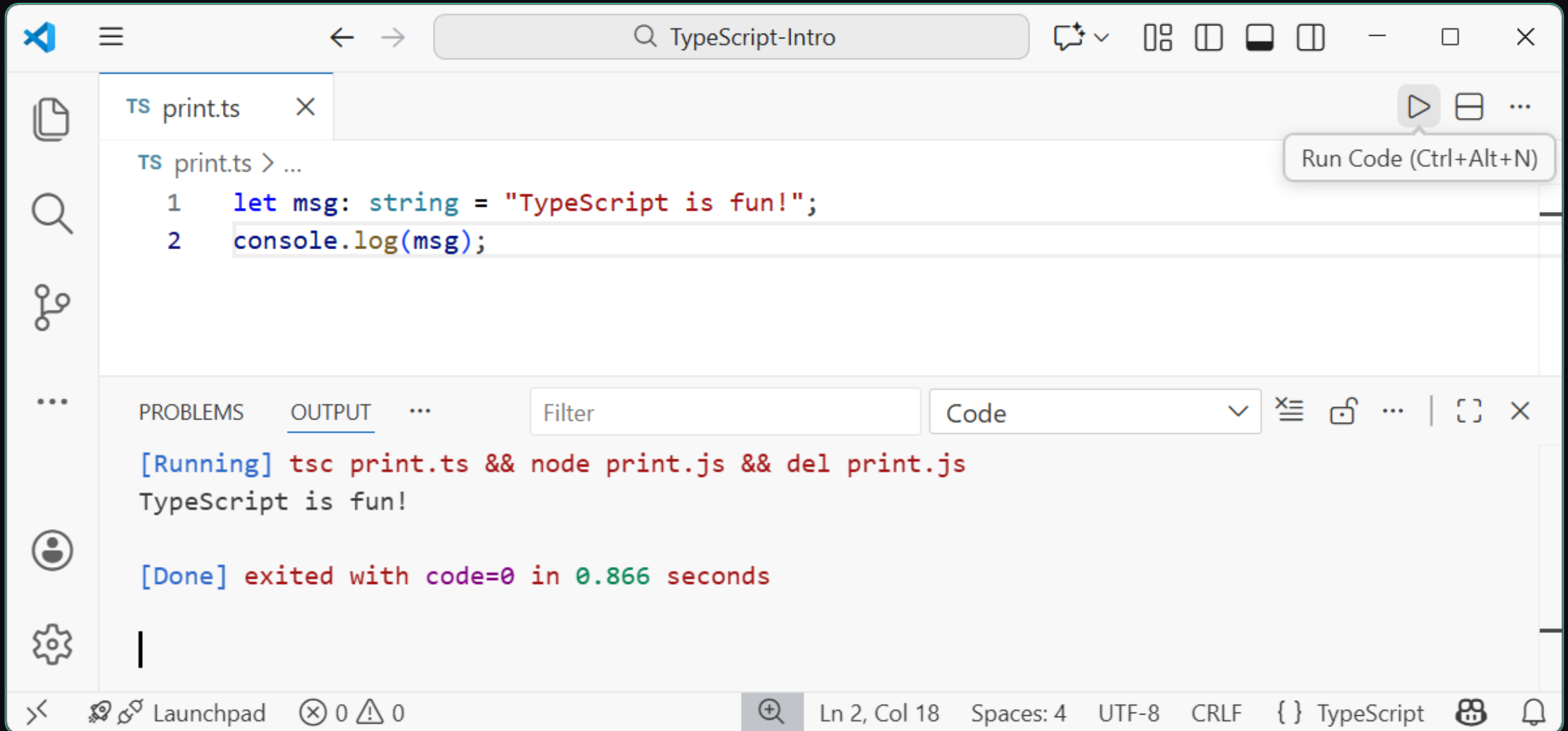
Configuring VS Code to Run TS

- **VS Code** cannot run directly **.ts** files
- Configure the **Code Runner** to run TS:



```
> AppData > Roaming > Code > User > {} settings.json > {} code-runner.executorMap > abc typescript
"code-runner.executorMap": {
  "csharp": "scriptcs",
  "vbscript": "cscript //nologo",
  "typescript": "tsc $fileName && node $fileNameWithoutExt.js && del $fileNameWithoutExt.js",
```

Code Runner in VS Code for TS



The screenshot shows the Visual Studio Code editor with a file named `print.ts` open. The code in the editor is:

```
1 let msg: string = "TypeScript is fun!";  
2 console.log(msg);
```

A tooltip for the `Run Code (Ctrl+Alt+N)` button is visible. Below the editor, the `OUTPUT` panel shows the execution results:

```
[Running] tsc print.ts && node print.js && del print.js  
TypeScript is fun!  
  
[Done] exited with code=0 in 0.866 seconds
```

The status bar at the bottom indicates the current position is `Ln 2, Col 18`, with `Spaces: 4`, `UTF-8` encoding, `CRLF` line endings, and the file is a `TypeScript` file.

TypeScript Execute (tsx)

- What is **tsx** (TypeScript Execute)?
 - A fast and simple way to execute TypeScript
- **Installing** TypeScript Execute (**tsx**):

```
npm -g install tsx
```

- **Running** TypeScript with **tsx**:

```
tsx example.ts
```



```
node example.js
```

- Note: **tsx** is not 100% compatible
 - It **skips type-checking**, so you still need **tsc**



Simple Types in TypeScript

- Simple **types** in TypeScript:

```
let age: number = 30;  
let name: string = "Alice";  
let isActive: boolean = true;  
let notDefined: undefined = undefined;  
let numberArr: number[] = [10, 20, 30];
```

- A special type "**any**" turns off type checking:

```
let whatever: any = 42;  
whatever = "Now I'm a string";
```


Type Inference and Type Checking

- **Type inference** in TypeScript:

```
let personName = "Alice"; // Valid → inferred type: string
personName = 42; // Type 'number' is not assignable to type 'string'
```

- Type **"unknown"** → check the type before use:

```
let input: unknown = "hello";
if (typeof input === "string") {
  console.log(input.toUpperCase()); // OK after check
}
console.log(input.toUpperCase()); // Error: Object type 'unknown'
```

Type Inference – Example

```
// TypeScript infers the shape of the object
```

```
const user = {  
  name: "Alice",    // Inferred → string  
  age: 30,          // Inferred → number  
  isAdmin: true     // Inferred → boolean  
};
```

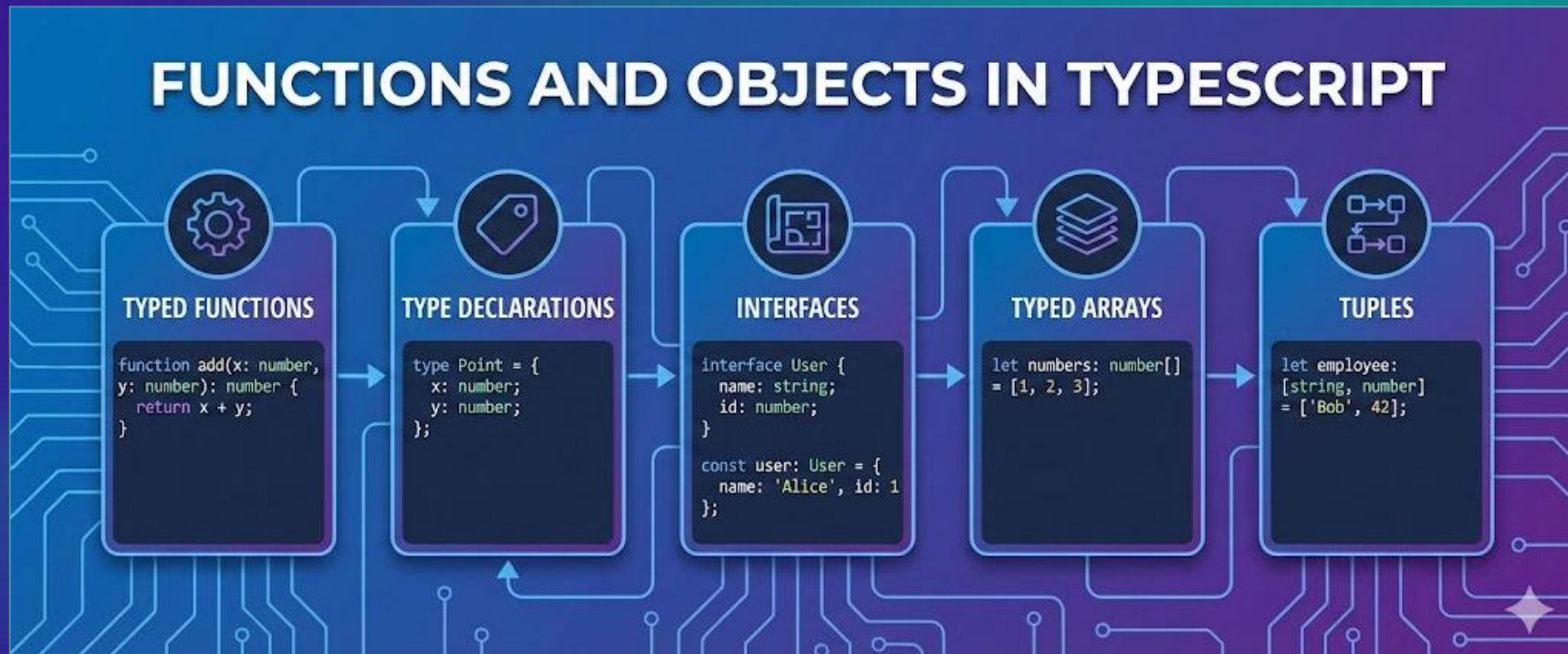
```
// TypeScript knows these properties exist
```

```
console.log(user.name, user.age); // OK
```

```
console.log(user.email); // Error: Property 'email' does not exist
```

Functions and Objects in TS

Typed Functions, Objects, Type Declarations, Interfaces, Typed Arrays, Tuples



Functions and Types

- Functions in TypeScript take **typed parameters** and return **typed result**:

```
function inchesToCentimeters(inches: number): number {  
    return inches * 2.54;  
}
```

- **Arrow functions** may have types:

```
const multiply =  
    (a: number, b: number): number => a * b;
```

Functions and Types (2)

- Functions **without return value** are declared **void**:

```
function printCurrentDate(): void {  
    let date = new Date();  
    console.log(date);  
}
```

- Function over **arrays**:

```
function sumNumbers(nums: number[]): number {  
    return nums.reduce((sum, x) => sum + x, 0);  
}
```

Objects in TypeScript

- **Objects** in TypeScript are typed:

```
let alice: { name: string; age: number } = {  
    name: "Alice",  
    age: 30  
};  
  
function printUser(  
    user: { name: string; age: number }): void {  
    console.log(`Name: ${user.name}, age: ${user.age}`);  
}  
  
printUser(alice);
```


Problem: Sort Students by Grades

- Write a **TypeScript function** to **sort** students by **grades**:

```
console.log(sortStudents(  
  {name: "Peter", grades: [5, 6, 6, 6, 5.60]},  
  {name: "Steve", grades: [5, 4, 4.50, 5, 5.30]},  
  {name: "Maria", grades: [6, 6, 5, 6]},  
  {name: "Clara", grades: [6, 6, 6, 5]}  
));  
  
[  
  { name: 'Clara', grades: [ 6, 6, 6, 5 ], avgGrade: 5.75 },  
  { name: 'Maria', grades: [ 6, 6, 5, 6 ], avgGrade: 5.75 },  
  { name: 'Peter', grades: [ 5, 6, 6, 6, 5.7 ], avgGrade: 5.74 },  
  { name: 'Steve', grades: [ 5, 4, 4.5, 5, 5.3 ], avgGrade: 4.76 }  
]
```

Solution: Sort Students by Grades

```
function sortStudents(...students: {name: string; grades: number[]}[]):  
    {name: string; grades: number[], avgGrade: number}[] {  
    let studs: {name: string; grades: number[]; avgGrade: number}[] = [];  
    for (const st of students) {  
        let s: {name: string; grades: number[]; avgGrade: number} =  
            {name: st.name, grades: st.grades, avgGrade: 0};  
        s.avgGrade = st.grades.reduce(  
            (sum, grade) => sum + grade, 0) / st.grades.length;  
        studs.push(s);  
    }  
    return studs.sort((a, b) => {  
        if (a.avgGrade !== b.avgGrade) {  
            return b.avgGrade - a.avgGrade; // Sort by average grade desc.  
        }  
        return a.name.localeCompare(b.name); // Sort by name ascending  
    });  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/typescript-intro/5281>

Type Declarations

- In TypeScript developers can declare **own data types**:

```
type Student = {  
  name: string;  
  grades: number[];  
  avgGrade?: number; // Optional property  
}  
  
let peter: Student = {name: "Peter", grades: [5, 6, 6, 6, 5.70]};  
console.log(peter);  
  
let anna: Student = {name: "Anna", grades: [5, 6], avgGrade: 5.50};  
console.log(anna);
```

Example: Sort Students by Grades

```
function sortStudents(...students: Student[]): Student[] {  
    let studs: Student[] = [];  
    for (const st of students) {  
        let s: Student = {name: st.name, grades: st.grades, avgGrade: 0};  
        s.avgGrade = st.grades.reduce(  
            (sum, grade) => sum + grade, 0) / st.grades.length;  
        studs.push(s);  
    }  
    return studs.sort((a, b) => {  
        if (a.avgGrade !== b.avgGrade)  
            return (b.avgGrade ?? 0) - (a.avgGrade ?? 0); // Sort by avgGrade  
        return a.name.localeCompare(b.name); // Sort by name ascending  
    });  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/typescript-intro/5281>

Optional and Read-Only Properties

```
type Person = {  
  name: string;  
  age: number;  
  isDeveloper?: boolean; // Optional property  
  readonly id: number; // Cannot be reassigned  
};  
  
const alice: Person = {  
  name: "Alice", age: 28, id: 123  
};  
  
alice.id = 999; // Error - readonly
```

- **Interfaces** are like type definitions, but more powerful:

```
interface IStudent {  
    name: string;  
    grades: number[];  
}  
  
interface IStudentWithGrade extends IStudent {  
    avgGrade: number;  
}  
  
let ann: IStudent = {name:"Ann", grades:[5, 6]};  
console.log(ann);  
  
let bob: IStudentWithGrade = {name:"Bob", grades:[6, 6, 5], avgGrade:5.67};  
console.log(bob);
```


Typed Arrays

- **Arrays** may also have types:

```
interface IStudent {  
    name: string;  
    grades: number[];  
}  
  
let students: IStudent[] = [  
    {name: "Maria", grades: [5, 6, 6, 5]},  
    {name: "Steve", grades: [6, 6, 6, 6, 6]},  
    {name: "Peter", grades: [5, 6, 5, 4, 5, 4]}  
];
```

Tuples (Fixed-Length Typed Array)

```
// Tuples (fixed length + known types)
let point: [number, number] = [10, 20];
let rgba: [number, number, number, number] = [255, 0, 0, 0.5];
console.log(point, rgba);
// [ 10, 20 ] [ 255, 0, 0, 0.5 ]
```

```
type TPerson = [string, number]; // [name, age]
let bob: TPerson = ["Bob", 25];
let carol: TPerson = ["Carol", 30];
console.log(bob, carol);
// [ 'Bob', 25 ] [ 'Carol', 30 ]
```

Problem: Sort Rectangles

- We are given **set of rectangles**, defined as their coordinates: $\{x1, y1\} - \{x2, y2\}$
 - Where $x1 < x2, y1 < y2$
- Define a data type **Rectangle**
 - Properties: **x1, y1, x2, y2**
- Write a JS function to **sort an array of rectangles** by their surface area

Example: Sort Rectangles

```
const rectangles: Rectangle[] = [  
  { x1: 0, y1: 0, x2: 5, y2: 4 }, // area: 20  
  { x1: 0, y1: 0, x2: 3, y2: 3 }, // area: 9  
  { x1: 5, y1: 0, x2: 10, y2: 2 }, // area: 10  
  { x1: 0, y1: 0, x2: 2, y2: 6 } // area: 12  
];  
console.log(sortRectanglesByArea(rectangles));
```

```
[  
  { x1: 0, y1: 0, x2: 3, y2: 3 },  
  { x1: 5, y1: 0, x2: 10, y2: 2 },  
  { x1: 0, y1: 0, x2: 2, y2: 6 },  
  { x1: 0, y1: 0, x2: 5, y2: 4 }  
]
```

Solution: Rectangles

```
interface Rectangle {  
  x1: number;  
  y1: number;  
  x2: number;  
  y2: number;  
}
```

```
function calculateArea(rect:  
  Rectangle): number {  
  const width = rect.x2 - rect.x1;  
  const height = rect.y2 - rect.y1;  
  return width * height;  
}
```

```
function sortRectanglesByArea(rects: Rectangle[]): Rectangle[] {  
  return rects.sort((a, b) => {  
    return calculateArea(a) - calculateArea(b);  
  });  
}
```

INVOICE

Invoice Number: 4000122771157

Date: 28.12.2023

Tax Event Date: 28.12.2023

Reference Document: 312531/23.12.2023

SUPPLIER

Vikiwat OOD

EIN: 160138003

VAT: BG160138003

4000 Plovdiv, ul. Ticha 17

CLIENT

NAKOV INNOVATIONS - EOOD

EIN: 204970307

VAT: BG204970307

Sofia, Bul. Aleksandar Malinov 78

LINE ITEMS

No.	Code	Description	Qty	Unit	Price	Disc %	Price w/ Disc	Total
1	146836	Floor socket XS Unica, INS52101	3	pcs	79.17	45%	43.54	130.63
2	3606	Power cable SVT 3x2.5mm ² , copper, grey, NYY	5	m	3.50	0%	3.50	17.50
3	100048	Courier service	1	pcs	7.99	0%	7.99	7.99

Subtotal: 156.12 BGN

VAT 20%: 31.22 BGN

Total: 187.34 BGN

In Words: One hundred eighty-seven lev and 34 stotinki

Invoices

Real-World Case Study

Problem: Invoices

- **Invoices** hold the following data: id, date, issuer, recipient, list of items, subtotal amount, VAT amount, total amount
- **Issuer company** holds the following data: name, id, tax id, address, manager, IBAN
- **Recipient company** holds the following data: name, id, tax id, address, manager
- **Invoice items** hold the following data: product, quantity, unit, price VAT rate, value (without VAT)
- Define a **TypeScript interface** to hold **invoices**

Sample Invoice



Invoice

INVOICE

Invoice Number: 4000122771157 Date: 28.12.2023
Tax Event Date: 28.12.2023 Reference Document: 312531/23

SUPPLIER

Vikiwat OOD
EIN: 160138003
VAT: BG160138003
4000 Plovdiv, ul. Ticha 17

LINE ITEMS

No.	Code	Description	Qty	Unit	Price	Disc %	Price w/ Disc	Total
1	146836	Floor socket XS Unica, INS52101	3	pcs	79.17	45%	43.54	130.63
2	3606	Power cable SVT 3x2.5mm ² , copper, grey, NYY	5	m	3.50	0%	3.50	17.50
3	100048	Courier service	1	pcs	7.99	0%	7.99	7.99

Subtotal: 156.12 BGN
VAT 20%: 31.22 BGN

Total: 187.34 BGN

In Words: One hundred eighty-seven lev and 34 stotinki

PAYMENT INFORMATION

Delivery Terms: CU-sender
Payment Terms: Wire transfer (Bank transfer)
Payment Due: 28.12.2023
Bank: Targovska banka D AD
IBAN: BG83DEMI92401000101947
BIC: DEMIBGSF

PAYMENTS

Date	Method	Amount
28.12.2023	Wire transfer	187.34 BGN
02.01.2024	Confirmed payment	187.34 BGN

Solution: Invoices

```
interface Invoice {  
  id: string;  
  date: Date;  
  issuer: IssuerCompany;  
  recipient: Company;  
  items: InvoiceItem[];  
  amountSubtotal: number;  
  vatTotal: number;  
  amountTotal: number;  
}
```

```
interface InvoiceItem {  
  product: string;  
  quantity: number;  
  unit?: string;  
  price: number;  
  vatRate: number;  
  valueNoVat: number;  
}
```

Solution: Invoices (2)

```
interface Company {  
    name: string;  
    id: string;  
    taxId?: string;  
    address?: string;  
    manager?: string;  
}
```

```
interface IssuerCompany extends Company {  
    iban: string; // IBAN is mandatory for issuer  
}
```

Problem: Invoice Validation

- Define "**Validation Result**" data structure to hold:
 - **isValid**, list of **errors** (if not valid)
- Write a function for **validate given invoice** data to find missing fields, wrong fields or incorrect calculations

```
let sampleInvoice: Invoice = { ... };  
let validationResult = validateInvoice(sampleInvoice);  
console.log("Is invoice valid?", validationResult.isValid);  
if (!validationResult.isValid) {  
    console.log("Errors:");  
    validationResult.errors.forEach(err => console.log("  ", err));  
}
```

Solution: Invoice Validation

- List of all possible **validations** to implement
 - Invoice cannot be empty.
 - Invoice ID is missing.
 - Invoice date is missing.
 - Invoice date cannot be in the future.
 - Issuer company information is missing.
 - Issuer company name is missing.
 - Issuer company ID is missing.
 - Issuer company IBAN is missing.
 - Issuer company IBAN format is invalid.
 - Recipient company information is missing.
 - Recipient company name is missing.
 - Recipient company ID is missing.
 - Invoice must have at least one item.
 - Item {index}: product name is missing.
 - Item {index}: unit is missing.
 - Item {index}: quantity must be greater than zero.
 - Item {index}: price cannot be negative.
 - Item {index}: VAT rate cannot be negative.
 - Item {index}: value calculation is incorrect.
 - Subtotal does not match the sum of items.
 - Total VAT cannot be negative.
 - Total VAT does not match sum of item VAT.
 - Total amount calculation is incorrect.

Unions, Literals, Enums

Unions, Literals, Enums, Type Aliases

TS

TypeScript Advanced Types

Union Types

`string | number`



Multiple Types
One Value

Literal Types

`"red" | "green" | "blue"`

`"red"` `"green"` `"blue"`

Exact Values
Type-Safe Constants

Enums

`enum Status { ... }`

`Pending = 0`

`Active = 1`

Named Constants

Type Aliases

`type User = { ... }`

`name: string`
`age: number`

Reusable Types

- **Unions** in TypeScript define **one of several types**

```
function padLeft(value: string | number,  
    padChar: string, length: number): string {  
    let strValue = value.toString();  
    while (strValue.length < length) {  
        strValue = padChar + strValue;  
    }  
    return strValue;  
}
```

```
console.log(padLeft("X2", "0", 7)); // "00000X2"  
console.log(padLeft(12.5, " ", 7)); // "    12.5"
```

Literal Types

- **Literal types** define a set of limited valid literals:

```
type Direction = "up" | "right" | "down" | "left";  
  
function move(dir: Direction) {  
    console.log(dir);  
}
```

```
move("down");    // OK  
move("west");    // Error
```

```
type StatusCode = 200 | 404 | 500;  
let code: StatusCode = 200;
```


- **Enums** declare a set of valid values

```
enum OrderStatus {  
    Pending = "Pending",  
    Processing = "Processing",  
    Shipped = "Shipped",  
    Delivered = "Delivered",  
    Cancelled = "Cancelled"  
}
```

```
let status: OrderStatus = OrderStatus.Pending;  
console.log(`Current order status: ${status}`);
```

Type Aliases

- **Type aliases** give another name for certain type definition

```
type ID = string | number;
```

```
type Name = string;
```

```
type Age = number;
```

```
type User = {
```

```
  id: ID;
```

```
  name: Name;
```

```
  age: Age;
```

```
};
```

```
let u: User = { id: 1, name: "Dave Parker", age: 21};
```

```
console.log(`User: ID: ${u.id}, Name: ${u.name}, Age: ${u.age}`);
```

Problem: Playing Cards

- Define a custom TypeScript type for **playing cards**
 - **CardFace**
 - **CardSuit**
 - **Card**
- Implement a function to **compare cards**
 - First, by **face**
 - Then by **suite**



Solution: Playing Cards

```
type CardFace = 'A' | '2' | '3' | '4' | '5' | '6' | '7' | '8'  
| '9' | '10' | 'J' | 'Q' | 'K';
```

```
type CardSuit = 'hearts' | 'diamonds' | 'clubs' | 'spades';
```

```
interface Card {  
    face: CardFace;  
    suit: CardSuit;  
};
```

```
function compareCards(card1: Card, card2: Card): number { ... }
```

Classes and Generics in TS

Classes, Types, Private Data, Generics

Classes

User

Properties:

- + name: string
- + email: string
- ◻ - password: string
- # role: string

Methods:

- + login(): boolean
- ◻ - validatePassword()
constructor(...)

Access Modifiers

- **public** Accessible everywhere
- ◻ **private** Class only
- **protected** Class + subclasses

Types

```
interface IUser {  
  name: string;  
  login(): boolean;  
}
```

Generics <T>

Reusable with any type

```
class Box<T> {  
  private value: T;  
  getValue(): T {  
    return this.value;  }  
}
```

Usage:

```
const stringBox = new Box<string>();
```

```
const numberBox = new Box<number>();
```

```
const userBox = new Box<User>();
```

Classes in TypeScript

- **Classes** in TypeScript define **data** + **operations**
- **Interfaces** are like classes, but without implementation
- **Objects** are class instances

```
interface Drawable {  
  x: number;  
  y: number;  
  draw(): string;  
}
```

```
class Circle implements Drawable {  
  public x: number;  
  public y: number;  
  private radius: number;  
  private color: string;  
  constructor(...) { implementation ... }  
  draw(): string { implementation ... }  
}
```

Problem: SVG Drawings

- **SVG** is a standard text-based format for vector graphic

```
<svg xmlns="http://www.w3.org/2000/svg" width="100" height="100">  
  <rect x="10" y="10" width="80" height="90" fill="green" />  
  <circle cx="50" cy="50" r="30" fill="lightblue" />  
</svg>
```

- We want to build **TypeScript classes** to draw objects in SVG format:
 - **Circle, Rectangle, Line**, etc.
 - **SVGDrawing** to hold a set of SVG objects

Solution: SVG Drawings

```
interface SVGObject {  
    x: number;  
    y: number;  
    drawAsSVG(): string;  
}
```

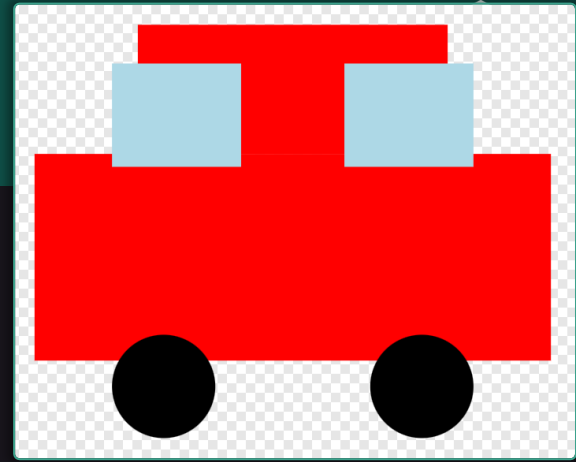
```
class Rectangle  
implements SVGObject { ... }
```

```
class Line  
implements SVGObject { ... }
```

```
class Circle implements SVGObject {  
    public x: number;  
    public y: number;  
    public radius: number;  
    public color: string;  
    constructor(...) { ... }  
    drawAsSVG() { ... }  
}
```

```
class SVGDrawing {  
    private objects: SVGObject[];  
}
```


Problem: SVG Drawings



- Define in TypeScript the following types:
 - Interface **SVGObject**: x, y, drawAsSVG()
 - Class **Circle**: x, y, radius, color, drawAsSVG()
 - Class **Rectangle**: x, y, width, height, color, drawAsSVG()
 - Class **Line**: x, y, x2, y2, color, drawAsSVG()
 - Class **SVGDrawing**, which holds a sequence of SVG objects
- Using the above classes, **draw a simple car** (rectangle, trapezoid, two windows and two wheels)
- Visualize the drawing in an HTML page

Generics in TypeScript

- **Generics** allow working with **multiple types**, while keeping **type-safety**

```
class Queue<T> {  
  private items: T[] = [];  
  enqueue(item: T) {  
    this.items.push(item);  
  }  
  dequeue(): T | undefined {  
    return this.items.shift();  
  }  
}
```

```
const q = new Queue<string>();  
q.enqueue("apple");  
q.enqueue("banana");  
console.log(q.dequeue());  
// apple  
console.log(q.dequeue());  
// banana
```

TypeScript and Vite

Using Vite for Web Projects

TypeScript & Vite

⚡ Vite

Lightning Fast

- Instant dev server
- Hot Module Reload
- Optimized builds

Modern Tools

- ES Modules native
- TypeScript ready
- Plugin ecosystem

Project Structure

```
my-vite-app/  
├── src/  
│   ├── main.ts  
│   ├── style.css  
│   └── counter.ts  
├── public/  
│   └── favicon.ico  
├── index.html  
├── package.json  
├── tsconfig.json  
├── vite.config.ts  
└── node_modules/
```

Quick Start Commands

Create Project:

```
$ npm create vite@latest my-app --template vanilla-ts
```

Install Dependencies:

```
$ npm install
```

Start Dev Server:

```
$ npm run dev
```

Build for Production:

```
$ npm run build
```



Build Faster •



Bundle Smarter •



Deploy with Confidence

What is Vite?

- To use HTML and TypeScript in a Web project, you need a **build tool**
- **Vite** is a **development server** and **build tool** for Web projects
 - Allows your code to be **served with HTTP** to Web browsers
 - Transforms **TypeScript** into JavaScript on demand
 - Allows **splitting large projects into components** (multiple HTML files, multiple JS files, multiple CSS files)
 - **Hot reload** on change (without restarting the server)
 - Can **build** static HTML files + scripts for deployment

- ```
npm create vite@latest my-app --template vanilla
```

```
PS C:\Projects\Intro-Programming-AI\TypeScript-Intro> npm create vite@latest my-app
```

```
> npx
> create-vite my-app
```

◆ Select a framework:

- Vanilla
- Vue
- React

◆ Select a framework:

Vanilla

◆ Select a variant:

- TypeScript
- JavaScript

# Creating a Project in Vite

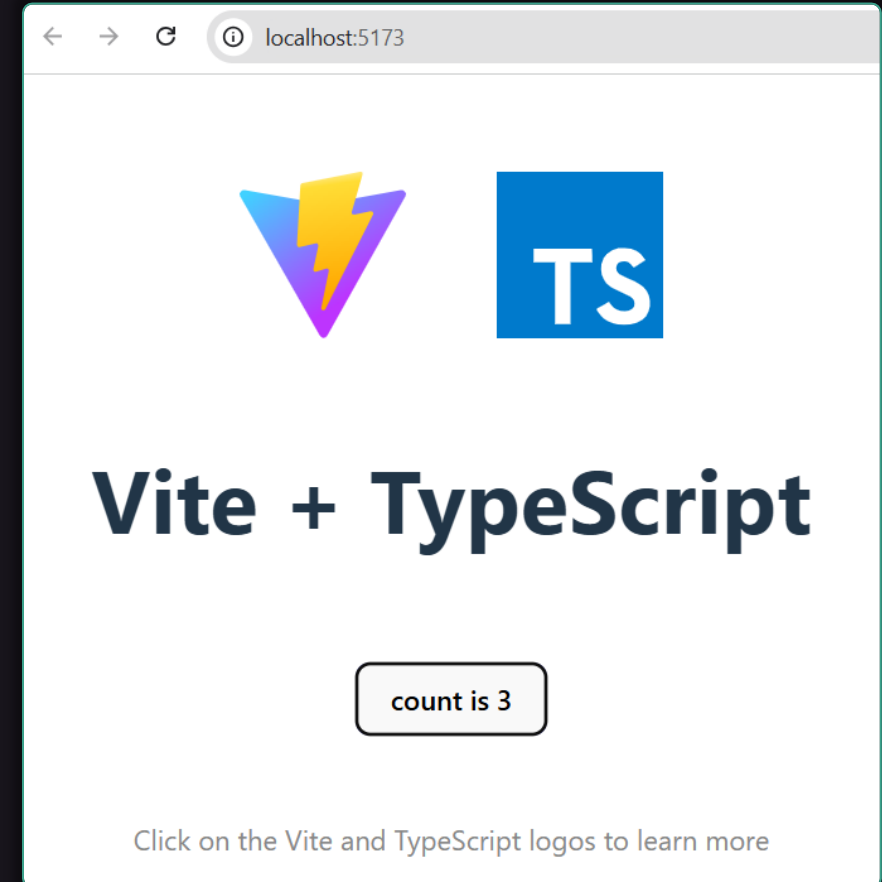
- Running a Vite project:

```
cd my-app
npm install
npm run dev
```

```
VITE v7.2.4 ready in 286 ms
```

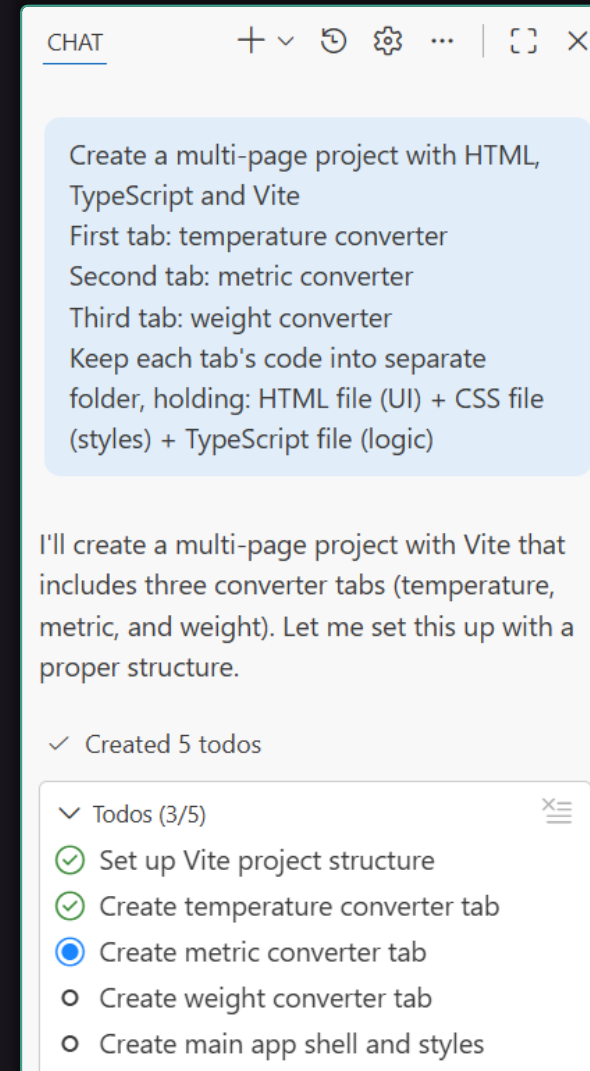
```
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

```
my-app
├── node_modules
├── public
│ └── vite.svg
├── src
│ ├── TS counter.ts
│ ├── TS main.ts
│ ├── # style.css
│ └── typescript.svg
├── .gitignore
├── index.html
├── package-lock.json
├── package.json
└── tsconfig.json
```



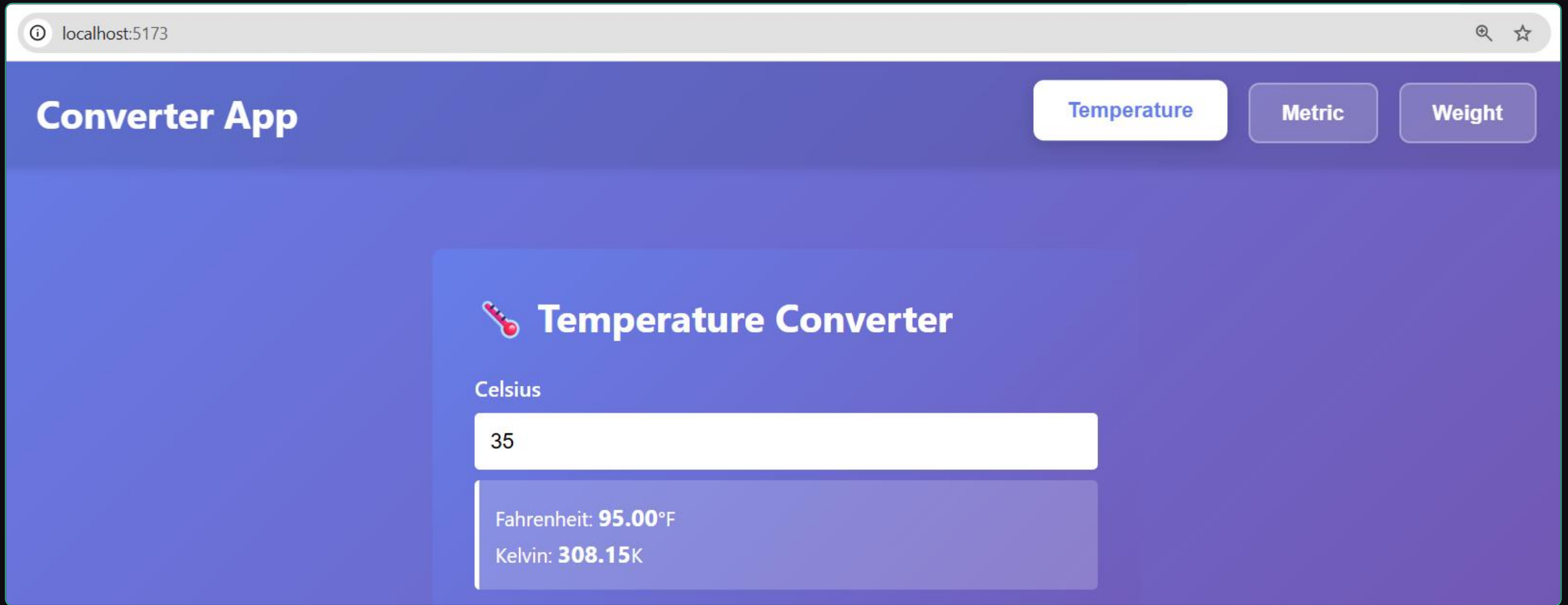
# Creating a Multi-Page Project

- Create a **multi-page Web project** with **HTML**, **TypeScript** and **Vite**
  - First tab: **temperature converter**
  - Second tab: **metric converter**
  - Third tab: **weight converter**
- Keep each tab's code into **separate folder**, holding: HTML file (UI) + CSS file (styles) + TypeScript file (logic)



# Multi-Page App in Vite

- This is an example of how the project can look like:





# Lesson Summary

- **TypeScript** = JavaScript + Types. Tools: **tsc** and **tsx**
- TypeScript allows defining typed variables, **typed functions**, **typed arrays**, **typed objects** and **tuples**
- **Unions**, **literal types** and **enums** allow defining types, which hold only **predefined values**, e.g. **"up" | "down"**
- **Classes** and **generics** implement fully-features OOP (object-oriented programming)
- In **Web projects** TypeScript is used with **Vite**: dev server, compiles TS on demand, allows creating components



# Questions?



# Postbank – Exclusive Partner for SoftUni AI



- One of the leading **banking institutions** in Bulgaria
- Member of the Eurobank Group with € 99.6 billion of assets
- Innovative trendsetter in the digitalization and transformation
- Certified Top Employer 2024 by the international Top Employers Institute
- AI integration for business, learning and development
- AI Assistants for talents: Story Builder, CV Assistant, Interview Trainer
- Proven people care and wellbeing initiatives
- Benefits and unlimited access to professional, personal and leadership trainings and programs
- [www.postbank.bg](http://www.postbank.bg) / [careers.postbank.bg](http://careers.postbank.bg)



# Diamond Partners of Software University



**SUPER  
HOSTING  
.BG**



**THE CROWN IS YOURS**



# Diamond Partners of SoftUni Digital



# Diamond Partners of SoftUni Digital



**HUMAN**

**IMPULSE MEDIA®**



**1FOR FIT**

**NETPEAK**  
DIGITAL GROWTH PARTNER

ABC DESIGN &  
COMMUNICATION

**Zahara**  
dig.it.all

**ETIENYANEV**  
Break Your *Limits*. Live Your *Brand*.





# Diamond Partners of SoftUni Creative



**SUPER  
HOSTING  
.BG**



# Organization Partners of SoftUni Creative



dequitas.

Фигмаўстор



Studio π

Think Big • Design Smart

дизайнът  
на нещата

REDV<sup>®</sup> PRINT  
STUDIO

THE  
BUCKS  
TOWN'S  
WORK